

Lab-four: Sharded KV Service

ShardCtrler实验记录

实验要求

ShardCtrler 是分片存储系统的“大脑”。它的核心任务是管理分片的分布情况，即决定哪个副本组（Replica Group）负责哪些数据分片。

配置

控制器必须维护一系列有序的 **Configuration (配置)**。每个配置包含：

- **Config Number:** 配置版本号（从 1 开始递增）。
- **Shards:** 一个长度为 10 的数组（实验固定分片数为 10），记录每个分片对应的 `GID`。
- **Groups:** 一个映射表，记录每个 `GID` 对应的服务器列表（`GID -> []ServerNames`）。
- **初始状态:** 第 0 号配置中，所有的 Shard 都分配给 `GID 0`（无效组），表示系统尚未就绪。

RPC 接口

`ShardCtrler` 是基于 Raft 的，所有操作必须具有**强一致性和去重处理**。

Join(Servers)

- **参数:** 一个从 `GID` 到服务器列表的映射。
- **动作:** 增加新的副本组。
- **要求:** 控制器必须创建一个新配置，将现有的分片尽可能均匀地重新分配给新的组。

Leave(GIDs)

- **参数:** 需要移除的 `GID` 列表。
- **动作:** 撤销指定的副本组。
- **要求:** 创建新配置，将原本属于这些组的分片分配给剩余的组，同样要保持负载均衡。

Move(Shard, GID)

- **参数:** 分片编号和目标组 ID。

- **动作：** 强制手动分配。
- **要求：** 将特定分片指派给某个组。这个操作主要用于调试或特殊策略，不需要考虑负载均衡。

Query(Num)

- **参数：** 配置版本号。
- **动作：** 查询特定版本的配置。
- **要求：** 如果 `Num` 是 -1 或大于当前最大版本号，则返回最新的配置。

Rebalance

当副本组发生变动（Join 或 Leave）时，要重新分配分片。

1. 平均分配
2. **确定性：** 只要输入的配置和变动相同，所有副本节点计算出的新配置必须完全一致。
3. **最小移动：** 减少分片的迁移次数。
4. **GID 0 处理：** 处理初始状态下从 GID 0 向真实组迁移的情况。

避坑

- **分片数组拷贝：** 每次创建新配置时，从旧配置中深拷贝（Deep Copy）数据，不能直接修改旧的 `Config` 对象。
- **排序保证确定性：** 在遍历 `Groups` 映射时，由于 Go 语言中 `map` 的遍历是随机的，必须先提取 `GID` 并进行**排序**，然后再进行均衡计算，否则不同节点的 Raft 状态机可能会产生不同的分片结果。
- **处理没有 Group 的情况：** 如果所有的组都 `Leave` 了，分片应该重新回到 `GID 0`。

实现细节

这部分主要是设计大脑控制下面的 `分布式shardkv数据库` 涉及到三个模块

- `common.go`：定义共享的数据结构（Config、RPC 参数/返回值、错误码、常量）。
- `client.go`：Clerk（客户端）逻辑，向 shard controller 集群发起 Join/Leave/Move/Query 请求，自动切换 leader。
- `server.go`：ShardCtrler 服务端，基于 Raft 实现多副本一致性，处理客户端请求，维护配置历史并执行负载均衡。

`common.go`：数据结构与协议定义

1. Config

- Num：配置号，单调递增（从 1 开始）。
- Shards：长度为 NShards=10 的数组，Shards[i] 表示 shard i 归属的 GID（0 表示未分配）。
- Groups：映射 gid -> servers[]，记录每个复制组的成员服务器地址。

2. RPC 参数/返回值

- JoinArgs / JoinReply：加入新 group。
- LeaveArgs / LeaveReply：离开一组 group。
- MoveArgs / MoveReply：手动指定某个 shard 的归属。
- QueryArgs / QueryReply：查询指定号或最新配置。

所有请求都带有 ClientId 和 SeqId 用于幂等去重；每个 reply 都包含 WrongLeader 标志和 Err。

3. 错误码

- OK、ErrWrongLeader、ErrTimeout。

client.go：客户端

Clerk 结构与初始化

- servers：所有 shard controller 服务器的 RPC 端点。
- clientId：全局唯一的客户端 ID（由 genClientId() 生成，利用时间戳+随机数+计数器）。
- seqId：单调递增的请求序号，用于去重。
- leaderId：最近一次通信成功的服务器索引，作为 leader 猜测，减少无效轮询。

请求重试与 leader 切换

每个操作（Join、Leave、Move、Query）都遵循相同的模式：

1. 构造请求参数（含 ClientId 和 SeqId）。
2. 向 ck.leaderId 指向的服务器发起 RPC。
3. 若成功且 !reply.WrongLeader && reply.Err == OK，返回结果。
4. 若回复 ErrTimeout（意味着请求可能已被 Raft 提交但未及时回复），继续重试同一台服务器。
5. 否则（WrongLeader 或 RPC 失败），切换到下一台服务器 (leaderId+1) % len(servers)，等待 50ms 后重试。

这样 Clerk 可以自动容错，在服务器崩溃或 leader 切换时仍然能完成操作。

Query 的特殊之处

Query 是只读操作，但为了线性一致性，服务端依然会通过 Raft 提交一个 Op（不修改状态），从而确保读取到的是已提交的配置。

server.go：服务端

ShardCtrler 结构

- rf：Raft 实例，用于复制日志。
- applyCh：Raft 提交的 ApplyMsg 通道。
- configs：配置历史切片，configs[0] 为初始空配置（Num=0），后续每执行一次 Join/Leave/Move 就追加一个新配置。
- dupTable：去重表，记录每个客户端最近一次写操作的结果，key 为 ClientId。
- notifyChans：等待 Raft apply 结果的 channel 映射，key 为 Raft 日志 index。

RPC Handler (Join/Leave/Move/Query)

每个 handler：

1. 构造 Op 结构体，包含操作类型、参数、ClientId、SeqId。
2. 调用 submitAndWait(op) 提交给 Raft 并等待应用完成。
3. 将结果返回给客户端。

submitAndWait – 提交并等待结果

1. 调用 rf.Start(op) 将命令写入 Raft 日志，获得 index。
2. 如果当前节点不是 leader，立即返回 ErrWrongLeader。
3. 为该 index 创建一个 channel 并注册到 notifyChans[index]（若同一 index 已有等待者，则触发旧的等待者返回 ErrWrongLeader）。
4. 用 select 等待 channel 上的结果，或超时（ApplyTimeout）返回 ErrTimeout。
5. 在函数退出前清理 notifyChans 中的对应条目。

applier – 异步应用已提交的命令

在一个独立 goroutine 中循环：

1. 从 applyCh 读取已提交的 ApplyMsg。
2. 反序列化出 Op。

3. **去重检查**：对于写操作（非 Query），若 `dupTable` 中已有相同 `clientId` 且 `seqId >=` 当前请求的 `seqId`，则直接返回缓存的结果，避免重复执行。
4. 若不是重复请求，调用 `applyOp(op)` 执行状态机操作，得到新结果，并更新去重表。
5. 找到对应 index 的 `notifyCh`，**在释放锁后**将结果发送给等待的 RPC handler，避免死锁。

状态机操作 `applyOp`

- **Join**：克隆最新配置，将新加入的 `servers` 合并到 `Groups`，然后调用 `rebalance` 重新分配 shard，生成新配置并追加到 `configs`。
- **Leave**：克隆最新配置，删除指定 GID 并把这些 GID 拥有的 shard 置为 0（临时无主），然后调用 `rebalance` 重新分配，生成新配置追加。
- **Move**：克隆最新配置，直接设置 `Shards[op.Shard] = op.GID`（不触发 `rebalance`），追加新配置。
- **Query**：根据 `Num` 返回对应配置（`-1` 或超出范围返回最新）。

负载均衡算法 `rebalance`

- 让所有 group 拥有的 shard 数量尽可能均匀（差值 ≤ 1 ）。
- 保持移动的 shard 数最小。
- 完全确定性（相同输入 $\rightarrow$ 相同输出），因此所有遍历都按 gid 排序。

步骤：

- a. 若无任何 group，所有 shard 归 0，返回。
- b. 对所有 gid 排序（确保确定性）。
- c. 计算 `avg = NShards / nGroups`，`extra = NShards % nGroups`（前 `extra` 个 group 多分 1 个 shard）。
- d. 统计每个 gid 当前拥有的 shard 列表（忽略 gid=0 的无主 shard）。
- e. 将无主 shard 和超额 group 的超额 shard 移入待分配池（pool）。
- f. 对不足目标数量的 group，从 pool 中按顺序取 shard 补足。
- g. 写回 `cfg.Shards`。

去重与线性一致性

- **读操作（Query）** 也会经过 Raft，保证了返回的配置是线性一致的——一定是某个时间点已提交的最新配置，而不会读到未提交的中间状态。
- **写操作**通过 `dupTable` 实现幂等：即使客户端因超时重发，服务器也只会执行一次状态改变，并返回相同的结果。

协同工作流程

1. 客户端调用 `Clerk.Join(servers)`。
2. Clerk 生成 `JoinArgs`，向它猜测的 leader 发送 RPC。
3. Leader 服务器的 `Join` handler 接收请求，构造 `Op{Type: OpJoin, Servers: ...}`，调用 `rf.Start(op)`。
4. Raft 将该 `Op` 复制到多数派节点，达成共识后推送至 `applyCh`。
5. `applier goroutine` 从 `applyCh` 取出已提交的 `Op`：
 - 检查去重表，若不是重复请求则执行 `applyOp`（执行 `rebalance`，生成新配置，追加到 `configs`）。
 - 更新去重表。
 - 通过 `notifyChans` 将结果发送给等待的 handler。
6. RPC handler 收到结果后，填充 `JoinReply`（`Err` 和 `WrongLeader`）返回给 Clerk。
7. Clerk 若得到 OK 则返回；否则根据错误类型重试或切换服务器。

后续任何客户端通过 `Query` 即可看到更新后的配置（线性一致性读）。这套机制保证了整个 shard controller 集群在面临网络分区、服务器崩溃时依然能正确、一致地管理配置，为上层 `shardkv` 提供可靠的配置变更通知。

common.go

```
1  package shardctrler
2
3  const NShards = 10
4
5  type Config struct {
6      Num      int           // config number
7      Shards [NShards]int // shard -> gid
8      Groups  map[int][]string // gid -> servers[]
9  }
10
11  const (
12      OK          = "OK"
13      ErrWrongLeader = "ErrWrongLeader"
14      ErrTimeout   = "ErrTimeout"
15  )
16
17  type Err string
18
19  // ===== Join =====
20
```

```
21  type JoinArgs struct {
22      Servers map[int][]string // new GID -> servers mappings
23      ClientId int64
24      SeqId    int64
25  }
26
27  type JoinReply struct {
28      WrongLeader bool
29      Err          Err
30  }
31
32  // ===== Leave =====
33
34  type LeaveArgs struct {
35      GIDs    []int
36      ClientId int64
37      SeqId    int64
38  }
39
40  type LeaveReply struct {
41      WrongLeader bool
42      Err          Err
43  }
44
45  // ===== Move =====
46
47  type MoveArgs struct {
48      Shard    int
49      GID       int
50      ClientId int64
51      SeqId     int64
52  }
53
54  type MoveReply struct {
55      WrongLeader bool
56      Err          Err
57  }
58
59  // ===== Query =====
60
61  type QueryArgs struct {
62      Num      int // desired config number, -1 = latest
63      ClientId int64
64      SeqId     int64
65  }
66
67  type QueryReply struct {
```

```
68     WrongLeader bool
69     Err          Err
70     Config       Config
71 }
72
```

```
client.go

1  package shardctrler
2
3  import (
4      "6.824/labrpc"
5      "crypto/rand"
6      "math/big"
7      "sync"
8      "sync/atomic"
9      "time"
10 )
11
12 type Clerk struct {
13     servers []*labrpc.ClientEnd
14     clientId int64
15     seqId    int64
16     leaderId int
17 }
18
19 // ===== clientId 生成器 =====
20 // 64bit: timestamp(32) | random(22) | counter(10)
21
22 var (
23     clientIdMu      sync.Mutex
24     clientIdLastSec int64
25     clientIdRand     int64
26     clientIdCounter  int64
27 )
28
29 func genClientId() int64 {
30     clientIdMu.Lock()
31     defer clientIdMu.Unlock()
32     now := time.Now().Unix()
33     if now != clientIdLastSec {
34         clientIdLastSec = now
35         clientIdRand = randBits(22)
36         clientIdCounter = 0
37     } else {
38         clientIdCounter = (clientIdCounter + 1) & 0x3FF
39     }
40     return (clientIdLastSec << 32) | clientIdRand << 10 | clientIdCounter
41 }
42
```



```

39     }
40     return (now << 32) | (clientIdRand << 10) | clientIdCounter
41 }
42
43 func randBits(bits uint) int64 {
44     max := big.NewInt(1)
45     max.Lsh(max, bits)
46     n, _ := rand.Int(rand.Reader, max)
47     return n.Int64()
48 }
49
50 func nrand() int64 {
51     max := big.NewInt(int64(1) << 62)
52     bigx, _ := rand.Int(rand.Reader, max)
53     return bigx.Int64()
54 }
55
56 func MakeClerk(servers []*labrpc.ClientEnd) *Clerk {
57     return &Clerk{
58         servers: servers,
59         clientId: genClientId(),
60         seqId: 0,
61         leaderId: 0,
62     }
63 }
64
65 func (ck *Clerk) nextSeq() int64 {
66     return atomic.AddInt64(&ck.seqId, 1)
67 }
68
69 // ===== Query =====
70 // Query 是只读操作, 走 Raft log 保证 linearizability
71
72 func (ck *Clerk) Query(num int) Config {
73     seq := ck.nextSeq()
74     args := &QueryArgs{
75         Num: num,
76         ClientId: ck.clientId,
77         SeqId: seq,
78     }
79     for {
80         var reply QueryReply
81         ok := ck.servers[ck.leaderId].Call("ShardCtrler.Query", args, &reply)
82         if ok && !reply.WrongLeader && reply.Err == OK {
83             return reply.Config
84         }
85         if ok && reply.Err == ErrTimeout {

```

```

86         // 超时: 可能已 apply, 继续等同一台
87         continue
88     }
89     ck.leaderId = (ck.leaderId + 1) % len(ck.servers)
90     time.Sleep(50 * time.Millisecond)
91 }
92 }
93
94 // ===== Join =====
95
96 func (ck *Clerk) Join(servers map[int][]string) {
97     seq := ck.nextSeq()
98     args := &JoinArgs{
99         Servers: servers,
100         ClientId: ck.clientId,
101         SeqId: seq,
102     }
103     for {
104         var reply JoinReply
105         ok := ck.servers[ck.leaderId].Call("ShardCtrler.Join", args, &reply)
106         if ok && !reply.WrongLeader && reply.Err == OK {
107             return
108         }
109         if ok && reply.Err == ErrTimeout {
110             continue
111         }
112         ck.leaderId = (ck.leaderId + 1) % len(ck.servers)
113         time.Sleep(50 * time.Millisecond)
114     }
115 }
116
117 // ===== Leave =====
118
119 func (ck *Clerk) Leave(gids []int) {
120     seq := ck.nextSeq()
121     args := &LeaveArgs{
122         GIDs: gids,
123         ClientId: ck.clientId,
124         SeqId: seq,
125     }
126     for {
127         var reply LeaveReply
128         ok := ck.servers[ck.leaderId].Call("ShardCtrler.Leave", args, &reply)
129         if ok && !reply.WrongLeader && reply.Err == OK {
130             return
131         }
132         if ok && reply.Err == ErrTimeout {

```

```

133         continue
134     }
135     ck.leaderId = (ck.leaderId + 1) % len(ck.servers)
136     time.Sleep(50 * time.Millisecond)
137 }
138 }
139
140 // ===== Move =====
141
142 func (ck *Clerk) Move(shard int, gid int) {
143     seq := ck.nextSeq()
144     args := &MoveArgs{
145         Shard:    shard,
146         GID:       gid,
147         ClientId: ck.clientId,
148         SeqId:     seq,
149     }
150     for {
151         var reply MoveReply
152         ok := ck.servers[ck.leaderId].Call("ShardCtrler.Move", args, &reply)
153         if ok && !reply.WrongLeader && reply.Err == OK {
154             return
155         }
156         if ok && reply.Err == ErrTimeout {
157             continue
158         }
159         ck.leaderId = (ck.leaderId + 1) % len(ck.servers)
160         time.Sleep(50 * time.Millisecond)
161     }
162 }
163

```

server.go

```

1  package shardctrler
2
3  import (
4      "6.824/labgob"
5      "6.824/labrpc"
6      "6.824/raft"
7      "sort"
8      "sync"
9      "sync/atomic"
10     "time"
11 )
12

```

```

13  const ApplyTimeout = 500 * time.Millisecond
14
15  // ===== Op =====
16
17  type OpType string
18
19  const (
20      OpJoin  OpType = "Join"
21      OpLeave  OpType = "Leave"
22      OpMove  OpType = "Move"
23      OpQuery OpType = "Query"
24  )
25
26  type Op struct {
27      Type      OpType
28      ClientId  int64
29      SeqId     int64
30
31      // Join
32      Servers map[int][]string
33
34      // Leave
35      GIDs []int
36
37      // Move
38      Shard int
39      GID   int
40
41      // Query
42      Num int
43  }
44
45  // ===== applyResult =====
46
47  type applyResult struct {
48      Err      Err
49      Config   Config // 仅 Query 使用
50      ClientId int64
51      SeqId    int64
52  }
53
54  // ===== 去重表 =====
55
56  type lastReply struct {
57      SeqId int64
58      Reply applyResult
59  }

```

```

60
61 // ===== notifyEntry =====
62
63 type notifyEntry struct {
64     clientId int64
65     seqId    int64
66     ch       chan applyResult
67 }
68
69 // ===== ShardCtrler =====
70
71 type ShardCtrler struct {
72     mu      sync.Mutex
73     me      int
74     rf      *raft.Raft
75     applyCh chan raft.ApplyMsg
76     dead    int32
77
78     configs []Config          // 配置历史, index 即 config num
79     dupTable map[int64]lastReply // 幂等性去重表
80     notifyChans map[int]notifyEntry
81 }
82
83 // ===== RPC Handler =====
84
85 func (sc *ShardCtrler) Join(args *JoinArgs, reply *JoinReply) {
86     op := Op{
87         Type:      OpJoin,
88         ClientId:    args.ClientId,
89         SeqId:       args.SeqId,
90         Servers:     args.Servers,
91     }
92     err, _ := sc.submitAndWait(op)
93     reply.Err = err
94     reply.WrongLeader = (err == ErrWrongLeader)
95 }
96
97 func (sc *ShardCtrler) Leave(args *LeaveArgs, reply *LeaveReply) {
98     op := Op{
99         Type:      OpLeave,
100         ClientId:    args.ClientId,
101         SeqId:       args.SeqId,
102         GIDs:        args.GIDs,
103     }
104     err, _ := sc.submitAndWait(op)
105     reply.Err = err
106     reply.WrongLeader = (err == ErrWrongLeader)

```

```

107 }
108
109 func (sc *ShardCtrler) Move(args *MoveArgs, reply *MoveReply) {
110     op := Op{
111         Type:      OpMove,
112         ClientId:   args.ClientId,
113         SeqId:      args.SeqId,
114         Shard:      args.Shard,
115         GID:        args.GID,
116     }
117     err, _ := sc.submitAndWait(op)
118     reply.Err = err
119     reply.WrongLeader = (err == ErrWrongLeader)
120 }
121
122 func (sc *ShardCtrler) Query(args *QueryArgs, reply *QueryReply) {
123     op := Op{
124         Type:      OpQuery,
125         ClientId:   args.ClientId,
126         SeqId:      args.SeqId,
127         Num:        args.Num,
128     }
129     err, cfg := sc.submitAndWait(op)
130     reply.Err = err
131     reply.WrongLeader = (err == ErrWrongLeader)
132     reply.Config = cfg
133 }
134
135 // ===== submitAndWait =====
136
137 func (sc *ShardCtrler) submitAndWait(op Op) (Err, Config) {
138     index, _, isLeader := sc.rf.Start(op)
139     if !isLeader {
140         return ErrWrongLeader, Config{}
141     }
142
143     ch := make(chan applyResult, 1)
144
145     sc.mu.Lock()
146     if old, exists := sc.notifyChans[index]; exists {
147         old.ch <- applyResult{Err: ErrWrongLeader}
148     }
149     sc.notifyChans[index] = notifyEntry{
150         clientId: op.ClientId,
151         seqId:     op.SeqId,
152         ch:        ch,
153     }

```

```

154     sc.mu.Unlock()
155
156     defer func() {
157         sc.mu.Lock()
158         if e, exists := sc.notifyChans[index]; exists &&
159             e.clientId == op.ClientId && e.seqId == op.SeqId {
160             delete(sc.notifyChans, index)
161         }
162         sc.mu.Unlock()
163     }()
164
165     select {
166     case result := <-ch:
167         if result.ClientId != op.ClientId || result.SeqId != op.SeqId {
168             return ErrWrongLeader, Config{}
169         }
170         return result.Err, result.Config
171     case <-time.After(ApplyTimeout):
172         return ErrTimeout, Config{}
173     }
174 }
175
176 // ===== applier =====
177
178 func (sc *ShardCtrler) applier() {
179     for !sc.killed() {
180         msg := <-sc.applyCh
181
182         if !msg.CommandValid {
183             continue
184         }
185
186         op, ok := msg.Command.(Op)
187         if !ok {
188             continue
189         }
190
191         sc.mu.Lock()
192
193         var result applyResult
194         result.ClientId = op.ClientId
195         result.SeqId = op.SeqId
196
197         // 去重 + 执行状态机, 用 isDup 替代 goto, 消除跳过变量声明的编译错误
198         isDup := false
199         if op.Type != OpQuery {

```

```

200         if last, dup := sc.dupTable[op.ClientId]; dup && last.SeqId >=
op.SeqId {
201             result.Err = last.Reply.Err
202             result.Config = last.Reply.Config
203             isDup = true
204         }
205     }
206     if !isDup {
207         // 执行状态机 (只取 Err/Config, 不覆盖 ClientId/SeqId)
208         r := sc.applyOp(op)
209         result.Err = r.Err
210         result.Config = r.Config
211         // 写操作更新去重表
212         if op.Type != OpQuery {
213             sc.dupTable[op.ClientId] = lastReply{SeqId: op.SeqId, Reply:
result}
214         }
215     }
216
217     var notifyCh chan applyResult
218     if e, exists := sc.notifyChans[msg.CommandIndex]; exists &&
e.clientId == op.ClientId && e.seqId == op.SeqId {
219         notifyCh = e.ch
220     }
221
222     sc.mu.Unlock()
223
224     // 锁外发送, 避免死锁
225     if notifyCh != nil {
226         notifyCh <- result
227     }
228 }
229 }
230 }
231
232 // ===== 状态机操作 =====
233
234 func (sc *ShardCtrler) applyOp(op Op) applyResult {
235     switch op.Type {
236
237     case OpJoin:
238         // 复制最新 config
239         newCfg := sc.cloneLatestConfig()
240         // 加入新 group
241         for gid, servers := range op.Servers {
242             newCfg.Groups[gid] = servers
243         }
244         // 重新均衡

```



```

245         rebalance(&newCfg)
246         sc.configs = append(sc.configs, newCfg)
247         return applyResult{Err: OK}
248
249     case OpLeave:
250         newCfg := sc.cloneLatestConfig()
251         // 把离开 group 的 shard 先归 0 (无主)
252         leaveSet := make(map[int]bool)
253         for _, gid := range op.GIDs {
254             leaveSet[gid] = true
255             delete(newCfg.Groups, gid)
256         }
257         for i, gid := range newCfg.Shards {
258             if leaveSet[gid] {
259                 newCfg.Shards[i] = 0
260             }
261         }
262         // 重新均衡
263         rebalance(&newCfg)
264         sc.configs = append(sc.configs, newCfg)
265         return applyResult{Err: OK}
266
267     case OpMove:
268         newCfg := sc.cloneLatestConfig()
269         newCfg.Shards[op.Shard] = op.GID
270         sc.configs = append(sc.configs, newCfg)
271         return applyResult{Err: OK}
272
273     case OpQuery:
274         n := op.Num
275         if n == -1 || n >= len(sc.configs) {
276             return applyResult{Err: OK, Config: sc.configs[len(sc.configs)-1]}
277         }
278         return applyResult{Err: OK, Config: sc.configs[n]}
279     }
280
281     return applyResult{Err: OK}
282 }
283
284 // cloneLatestConfig 深拷贝最新 config, 并将 Num+1
285 func (sc *ShardCtrler) cloneLatestConfig() Config {
286     latest := sc.configs[len(sc.configs)-1]
287     newCfg := Config{
288         Num:    latest.Num + 1,
289         Shards: latest.Shards,
290         Groups: make(map[int][]string),
291     }

```

```

292     for gid, servers := range latest.Groups {
293         cp := make([]string, len(servers))
294         copy(cp, servers)
295         newCfg.Groups[gid] = cp
296     }
297     return newCfg
298 }
299
300 // ===== 负载均衡算法 =====
301 //
302 // 目标:
303 // 1. 每个 group 分到的 shard 数尽量相等 (差值 <= 1)
304 // 2. 移动的 shard 数尽量少
305 // 3. 结果必须确定性 (相同输入 -> 相同输出), 因此所有遍历按 gid 排序
306 //
307 // 算法:
308 // avg = NShards / nGroups - 每个 group 至少分到 avg 个
309 // extra = NShards % nGroups - 前 extra 个 group 多分 1 个
310 //
311 // 1. 统计每个 gid 当前持有的 shard 列表
312 // 2. 计算每个 gid 目标数量 (avg 或 avg+1)
313 // 3. 把 "持有超额" 的 shard 放入待分配池
314 // 4. 把 "持有不足" 的 gid 从池中取 shard
315
316 func rebalance(cfg *Config) {
317     nGroups := len(cfg.Groups)
318     if nGroups == 0 {
319         // 没有 group, 所有 shard 归 0
320         for i := range cfg.Shards {
321             cfg.Shards[i] = 0
322         }
323         return
324     }
325
326     // 按 gid 排序, 保证确定性
327     gids := sortedGIDs(cfg.Groups)
328
329     avg := NShards / nGroups
330     extra := NShards % nGroups // 前 extra 个 gid 分 avg+1
331
332     // 统计每个 gid 当前拥有的 shard (按 shard index 排序保证确定性)
333     owned := make(map[int][]int) // gid -> []shardIndex
334     for _, gid := range gids {
335         owned[gid] = []int{}
336     }
337     // gid=0 的 shard (无主) 也需要分配出去
338     var unassigned []int

```

```

339     for shard, gid := range cfg.Shards {
340         if gid == 0 || owned[gid] == nil {
341             unassigned = append(unassigned, shard)
342         } else {
343             owned[gid] = append(owned[gid], shard)
344         }
345     }
346
347     // 把超额的 shard 归入待分配池
348     // 先处理分配 avg+1 的 group, 再处理分配 avg 的 group
349     pool := unassigned // 待分配池
350
351     for i, gid := range gids {
352         target := avg
353         if i < extra {
354             target = avg + 1
355         }
356         shards := owned[gid]
357         if len(shards) > target {
358             // 超额: 把多余的放入 pool (从末尾取, 保证确定性)
359             pool = append(pool, shards[target:]...)
360             owned[gid] = shards[:target]
361         }
362     }
363
364     // 从 pool 补充不足的 group
365     for i, gid := range gids {
366         target := avg
367         if i < extra {
368             target = avg + 1
369         }
370         for len(owned[gid]) < target {
371             owned[gid] = append(owned[gid], pool[0])
372             pool = pool[1:]
373         }
374     }
375
376     // 写回 cfg.Shards
377     for gid, shards := range owned {
378         for _, shard := range shards {
379             cfg.Shards[shard] = gid
380         }
381     }
382 }
383
384 // sortedGIDs 返回按升序排列的 gid 列表, 保证遍历确定性
385 func sortedGIDs(groups map[int][]string) []int {

```

```

386     gids := make([]int, 0, len(groups))
387     for gid := range groups {
388         gids = append(gids, gid)
389     }
390     sort.Ints(gids)
391     return gids
392 }
393
394 // ===== Kill =====
395
396 func (sc *ShardCtrler) Kill() {
397     atomic.StoreInt32(&sc.dead, 1)
398     sc.rf.Kill()
399 }
400
401 func (sc *ShardCtrler) killed() bool {
402     return atomic.LoadInt32(&sc.dead) == 1
403 }
404
405 func (sc *ShardCtrler) Raft() *raft.Raft {
406     return sc.rf
407 }
408
409 // ===== StartServer =====
410
411 func StartServer(servers []*labrpc.ClientEnd, me int, persister
    *raft.Persister) *ShardCtrler {
412     labgob.Register(Op{})
413
414     sc := &ShardCtrler{
415         me:      me,
416         dupTable: make(map[int64]lastReply),
417         notifyChans: make(map[int]notifyEntry),
418     }
419
420     sc.configs = make([]Config, 1)
421     sc.configs[0].Groups = map[int][]string{}
422
423     sc.applyCh = make(chan raft.ApplyMsg, 64)
424     sc.rf = raft.Make(servers, me, persister, sc.applyCh)
425
426     go sc.applier()
427
428     return sc
429 }

```

ShardKV实验记录

实验要求

ShardKV实现一个能够感知配置变化、自动迁移数据、且在迁移过程中依然保证线性一致性的分布式数据库。

分片存储与服务

每个 `ShardKV` 副本组（Replica Group）只负责处理属于它的分片。

- **分片过滤**：当客户端请求（Get/Put/Append）到达时，Server 必须检查该 Key 所属的 Shard 是否由本组负责。
- **ErrWrongGroup**：如果请求的 Key 不在本组负责范围内，必须返回错误，告知客户端去更新配置并重试。

配置更新与数据迁移

当 `shardctrler` 的配置发生变化时，系统必须在不停机的情况下完成分片所有权的转移。

配置检测

Server 需要启动一个后台线程，定期向 `shardctrler` 查询最新配置。

- **循序渐进**：必须等当前配置下所有的迁移任务都完成后，才能更新到下一个配置（一次只处理一个配置变更）。

数据拉取

如果新配置显示 Group A 得到了原本属于 Group B 的 Shard 10：

1. Group A 必须主动向 Group B 发起 RPC 请求（如 `GetShardData`）。
2. Group B 在发现配置更新后，必须**停止**对 Shard 10 的服务，并将该分片的数据打包。
3. Group A 接收到数据并提交到自己的 Raft 日志中，应用后才正式接管该分片。

保证

A. 线性一致性

即使在迁移过程中，对于任何一个 Key 的操作也必须看起来是原子且即时的。

- **状态同步**：数据迁移不仅要迁移 Key-Value 对，还要迁移**客户端去重表（Duplicate Detection Table）**。否则，迁移后客户端重试之前的请求可能会被重复执行。

B. 垃圾回收

当 Group A 确认已经成功接收了 Shard 10 后，Group B 必须删除其本地存储的 Shard 10 数据，以释放空间。

- **安全删除：** 不能在发送完数据后立即删除，必须收到 Group A 的“确认收货”反馈后才能在 Raft 状态机中清理。

C. 空日志追加

当一个组新当选 Leader 且面临配置变更时，需要提交一个空日志（No-op）来确定当前 commit index，从而避免在配置转换期间出现活锁。

避坑

- **死锁（Deadlock）：** 配置更新线程、数据拉取线程和 Raft 状态机应用线程之间极易发生锁竞争。建议尽量缩小锁的粒度。
- **状态机阻塞：** 如果 Raft 正在处理配置变更（等待拉取数据），它不能阻塞其他不相关分片的处理（虽然实验中为了简单通常会整体暂停，但优化版需要分片级锁定）。
- **网络分区下的迁移：** 如果 Group A 和 Group B 之间网络断开，迁移会卡住。你的系统必须能够处理这种不确定性，并在网络恢复后自动继续。



实现细节

整个 `shardkv` 系统依赖两个核心组件：

- **shardctrler**（分片控制器）：管理配置（config），决定每个 shard 属于哪个 group。
- **Raft**：在每个 group 内部提供复制和一致性。

`shardkv` 本身实现了：

- `common.go` 提供了通信的“语言”，确保客户端、服务器、迁移协议统一理解数据结构。
- `client.go` 封装了路由、重试和配置刷新，使上层应用不必关心底层分片细节。
- `server.go` 利用 Raft 实现了强一致的状态机，并通过精心设计的状态机和三个后台协程，将复杂的 shard 迁移过程拆解为可控制的步骤，保证了动态配置下 KV 存储的正确性。

流程

Clerk → 查询 config → 定位 gid → 向该 group 的 leader 发送 Get/PutAppend RPC



ShardKV server ← Raft 复制命令 → apply → 执行状态机（包括迁移）



configPoller → 查询 shardctrler 获取新 config → Raft 提交 OpConfig

↓

migrationPuller → PullShard RPC → OpInstall

↓

gcHandler → GCShard RPC → OpGC

common.go：协议与数据类型

错误码

覆盖所有可能的结果，`ErrWrongGroup` 用于告知客户端 key 不属于当前 group，触发配置刷新。

`OK`, `ErrNoKey`, `ErrWrongGroup`, `ErrWrongLeader`, `ErrTimeout`, `ErrNotReady`

客户端 KV 请求

- `PutAppendArgs` / `PutAppendReply`：写请求，包含 `Op` 字段区分 `Put` 和 `Append`。
- `GetArgs` / `GetReply`：读请求。
- 所有请求都带 `ClientId` + `SeqId`，用于服务端幂等去重。

去重表条目

`DupEntry` 记录一次写操作的结果（`SeqId` + `Err` + `Value`）。在 shard 迁移时随数据一起传输，保证新 owner 也能识别重复请求。

Shard 迁移 RPC

- `PullShardArgs` / `PullShardReply`：新 owner 从旧 owner 拉取指定 shard 的全部 KV 数据及去重表。
- `GCArgs` / `GCReply`：新 owner 通知旧 owner 删除已迁移的 shard 数据。

client.go：客户端

结构与初始化

- `sm`：shardctrler 的 Clerk，用于查询最新配置。
- `config`：缓存的最新配置，确定 key 所在的 gid 及服务器列表。
- `make_end`：函数，用于创建到任意服务器的 RPC 连接。

- `clientId`：全局唯一，由 `genClientId()` 利用时间戳+随机数+计数器生成。
- `seqId`：原子递增，为每个请求生成唯一序号。
- `leaderIds`：缓存每个 gid 上一次通信成功的服务器索引，避免每次都从头轮询。

初始化时，`MakeClerk()` 立刻调用 `sm.Query(-1)` 获取最新配置。

请求路由与重试逻辑

`Get` 和 `PutAppend` 都遵循相同的循环模式：

1. 计算 key 所在的 shard： `key2shard(key)`。
2. 从配置中查出 `gid = config.Shards[shard]`。
3. 若该 gid 有服务器列表：
 - 从缓存的 `leaderIds[gid]` 开始尝试发送 RPC。
 - 如果成功（`OK` 或 `ErrNoKey`），更新缓存并返回。
 - 如果收到 `ErrWrongGroup`，说明该 group 已不负责此 shard（可能发生了配置变更），**退出内层循环**，刷新配置后重试。
 - 如果是 `ErrWrongLeader` / `ErrTimeout` / RPC 失败，尝试该 group 的下一台服务器。
4. 若该 gid 没有服务器，或内层循环全部失败，休眠 100ms 后重新查询配置并重试。

这种设计使得客户端能够自动适应配置变化、leader 切换和服务器重启。

`server.go`：服务端

Shard 状态机

每个 shard 在服务器内部处于四种状态之一，由设计说明中的注释描述：

Serving —— config change ——► BePulling (旧 owner 停止服务，等待新 owner 拉取)

Serving ◀—— OpGC applied —— BePulling

(新 owner 无数据) —— config change ——► Pulling (需要从旧 owner 拉取)

Pulling —— OpInstall ——► GCing (已有数据可服务，等待通知旧 owner GC)

GCing —— OpGC applied ——► Serving (GC 完成，稳定)

Config 推进原则：只有当所有 shard 都恢复到 `Serving` 状态时，才允许推进到下一个 config，保证同一时间只有一个配置的迁移在进行。

RPC Handler

所有对外 RPC 最终都通过 `submitOp` 将 Op 提交给 Raft，然后等待 apply。

- **Get / PutAppend：**构造 `Op{Type, Key, Value, ClientId, SeqId}`，调用 `submitOp`。
- **PullShard**（被旧 owner 响应）：
 - 仅 leader 处理。
 - 检查自身 config 版本是否足够新（`>= args.ConfigNum`），否则返回 `ErrNotReady`。
 - 把请求的 shard 的 KV 数据和 **完整去重表** 返回给新 owner。
 - 此操作只读，不经过 Raft（直接返回 leader 的内存状态）。
- **GCShard**（被旧 owner 响应）：
 - 检查重复 GC 或已完成 GC 的情况，避免重复提交。
 - 若确实需要 GC，提交一个 `OpGC` 操作给 Raft，稍后 apply 时真正清空 shard 数据。

`submitOp`：统一提交流程

1. 生成随机 `InternalId` 作为本次提交的唯一标识（用于匹配 apply 结果）。
2. 调用 `rf.Start(op)` 将命令写入 Raft 日志。
3. 如果不是 leader，立即返回 `ErrWrongLeader`。
4. 为该 index 创建一个带 buffer 的 channel，并注册到 `notifyChans` 中。如果该 index 已有旧的等待者（leader 变更、日志复写），先通知它失败。
5. 使用 `select` 等待 channel 结果或超时（`ApplyTimeout` → `ErrTimeout`）。
6. 函数退出前清理 `notifyChans` 中的对应条目。

`applier` goroutine：应用已提交命令

- 从 `applyCh` 读取 Raft 提交的消息。
- 若是 **Snapshot**：调用 `CondInstallSnapshot` 安装，若接受则反序列化快照。
- 若是 **Command**：
 - a. 类型断言为 `Op`。
 - b. 加锁后：

- 对于 **KV op**：先检查 ownership 和 shard 状态（`ownerGid != kv.gid` 或状态非 `Serving/GCing` 则返回 `ErrWrongGroup`）。
 - 写操作：检查去重表，若重复则返回缓存结果；否则执行 `applyKVOp` 并更新去重表。
 - 读操作：直接执行 `applyKVOp`（读是幂等的，无需去重）。
 - 对于 **管理 op**（Config/Install/GC）：调用 `applyAdminOp` 执行状态转换。
- c. 取出对应 index 的 `notifyCh`（在锁外发送，防止死锁）。
- d. 若 Raft 状态过大且 `maxraftstate != -1`，创建快照（序列化 shards、dupTable、config、lastConfig），调用 `doSnapshot`（内部调用 `rf.Snapshot`）。
- e. 释放锁后，将结果通过 channel 发送给等待的 RPC handler。

状态机核心： `applyAdminOp`

- **OpConfig**：仅当 `op.Config.Num == kv.config.Num+1` 时接受。保存 `lastConfig`，更新 `config`，然后根据新旧配置比较每个 shard 的所有权变化，设置相应的状态（`Serving` → `Pulling` / `BePulling`）。
- **OpInstall**：只接受与当前 config 匹配的安装请求。将接收到的 KV 数据写入对应 shard，并将状态从 `Pulling` 转为 `GCing`。同时合并去重表（取较大 SeqId）。
- **OpGC**：根据当前状态和配置号执行删除或转换：
 - 旧 owner 的 `BePulling` → 清空数据，变为 `Serving`。
 - 新 owner 的 `GCing` → 变为 `Serving`。

后台 goroutine

三个后台任务仅在当前节点是 leader 时运行：

- **configPoller**（每 100ms）：

检查是否所有 shard 都是 `Serving`，若是则向 `shardctrler` 查询下一个配置，若存在则提交 `OpConfig`。

- **migrationPuller**（每 50ms）：

查找所有处于 `Pulling` 状态的 shard，按旧 owner 的 gid 分组，并发调用 `PullShard` RPC 拉取数据，成功后提交 `OpInstall`。

- **gcHandler**（每 50ms）：

查找所有处于 `GCing` 状态的 shard，并发向旧 owner 发送 `GCSHard` RPC，成功后提交 `OpGC` 标记自己进入 `Serving`。

快照与恢复

- `doSnapshot`：用 gob 编码当前 shards、dupTable、config、lastConfig，调用 `rf.Snapshot` 持久化。
- `installSnapshot`：从快照数据中解码出上述四个字段，直接恢复内存状态。

协同 workflow 举例：配置变更导致 shard 迁移

1. 管理员通过 `shardctrler` 发起 Join/Leave，`shardctrler` 将新配置写入其 Raft 日志并 apply。
2. `ShardKV` 的 `configPoller` 轮询 `shardctrler`，发现新 config ($\text{Num} = N+1$)，提交 `OpConfig` 给 Raft。
3. 各 group 通过 Raft 复制并 **apply OpConfig**:
 - 旧 owner 将对应 shard 标记为 `BePulling`，停止服务该 shard。
 - 新 owner 将对应 shard 标记为 `Pulling`，开始等待数据。
4. 新 owner 的 `migrationPuller` 检测到 `Pulling` 状态，向旧 owner 发送 `PullShard` RPC。
5. 旧 owner 返回该 shard 的 KV 数据及去重表。
6. 新 owner 收到数据后，提交 `OpInstall` 给 Raft，apply 时将数据写入 shard，状态变为 `GCing`（此时已经可以服务客户端请求，但读写在允许的状态范围内）。
7. 新 owner 的 `gcHandler` 检测到 `GCing`，向旧 owner 发送 `GCSHard` RPC。
8. 旧 owner 收到 `GCSHard` 后，提交 `OpGC`，apply 时清空该 shard 数据，状态变回 `Serving`。
9. 新 owner 收到旧 owner 的 GC 确认后，也提交自己的 `OpGC`，状态变回 `Serving`。
10. 所有 shard 都回到 `Serving` 后，`configPoller` 才能拉取下一版配置，防止级联迁移出错。

整个过程保证了线性一致性、容错和数据不丢失。

common.go

```
1 package shardkv
2
3 const (
4     OK          = "OK"
5     ErrNoKey    = "ErrNoKey"
6     ErrWrongGroup = "ErrWrongGroup"
```

```

7     ErrWrongLeader = "ErrWrongLeader"
8     ErrTimeout     = "ErrTimeout"
9     ErrNotReady    = "ErrNotReady"
10 )
11
12 type Err string
13
14 // ===== 客户端 KV ops =====
15
16 type PutAppendArgs struct {
17     Key      string
18     Value    string
19     Op       string // "Put" or "Append"
20     ClientId int64
21     SeqId    int64
22 }
23
24 type PutAppendReply struct {
25     Err Err
26 }
27
28 type GetArgs struct {
29     Key      string
30     ClientId int64
31     SeqId    int64
32 }
33
34 type GetReply struct {
35     Err  Err
36     Value string
37 }
38
39 // ===== 去重表条目 (跨 group 传输) =====
40
41 // DupEntry 放 common.go, 因为 PullShardReply 需要它
42 type DupEntry struct {
43     SeqId int64
44     Err   Err
45     Value string
46 }
47
48 // ===== Shard 迁移 RPC =====
49
50 // PullShard: 新 owner 向旧 owner 拉取 shard 数据
51 type PullShardArgs struct {
52     ConfigNum int
53     ShardIds  []int

```

```

54 }
55
56 type PullShardReply struct {
57     Err      Err
58     Shards   map[int]map[string]string // shardId -> kv
59     DupTable map[int64]DupEntry      // 随 shard 一起迁移, 保证幂等性
60 }
61
62 // GCShard: 新 owner 通知旧 owner 删除已迁移的 shard 数据
63 type GCArgs struct {
64     ConfigNum int
65     ShardIds  []int
66 }
67
68 type GCReply struct {
69     Err Err
70 }

```

```

client.go
1  package shardkv
2
3  import (
4      "6.824/labrpc"
5      "6.824/shardctrler"
6      "crypto/rand"
7      "math/big"
8      "sync"
9      "sync/atomic"
10     "time"
11 )
12
13 func key2shard(key string) int {
14     shard := 0
15     if len(key) > 0 {
16         shard = int(key[0])
17     }
18     shard %= shardctrler.NShards
19     return shard
20 }
21
22 // ===== clientId 生成器 =====
23 // 64bit: timestamp(32) | random(22) | counter(10)
24
25 var (
26     clientIdMu      sync.Mutex

```

```

27     clientIdLastSec int64
28     clientIdRand    int64
29     clientIdCounter int64
30 )
31
32 func genClientId() int64 {
33     clientIdMu.Lock()
34     defer clientIdMu.Unlock()
35     now := time.Now().Unix()
36     if now != clientIdLastSec {
37         clientIdLastSec = now
38         clientIdRand = randBits(22)
39         clientIdCounter = 0
40     } else {
41         clientIdCounter = (clientIdCounter + 1) & 0x3FF
42     }
43     return (now << 32) | (clientIdRand << 10) | clientIdCounter
44 }
45
46 func randBits(bits uint) int64 {
47     max := big.NewInt(1)
48     max.Lsh(max, bits)
49     n, _ := rand.Int(rand.Reader, max)
50     return n.Int64()
51 }
52
53 func nrand() int64 {
54     max := big.NewInt(int64(1) << 62)
55     n, _ := rand.Int(rand.Reader, max)
56     return n.Int64()
57 }
58
59 // ===== Clerk =====
60
61 type Clerk struct {
62     sm      *shardctrler.Clerk
63     config  shardctrler.Config
64     make_end func(string) *labrpc.ClientEnd
65
66     clientId int64
67     seqId    int64
68     mu       sync.Mutex
69     leaderIds map[int]int // gid -> 上次成功的 server index, 减少轮询
70 }
71
72 func MakeClerk(ctrlers []*labrpc.ClientEnd, make_end func(string)
    *labrpc.ClientEnd) *Clerk {

```

```

73     ck := &Clerk{
74         sm:      shardctrler.MakeClerk(ctrlers),
75         make_end: make_end,
76         clientId: genClientId(),
77         leaderIds: make(map[int]int),
78     }
79     ck.config = ck.sm.Query(-1)
80     return ck
81 }
82
83 func (ck *Clerk) nextSeq() int64 {
84     return atomic.AddInt64(&ck.seqId, 1)
85 }
86
87 // ===== Get =====
88
89 func (ck *Clerk) Get(key string) string {
90     seq := ck.nextSeq()
91     args := GetArgs{
92         Key:      key,
93         ClientId: ck.clientId,
94         SeqId:     seq,
95     }
96
97     for {
98         shard := key2shard(key)
99         gid := ck.config.Shards[shard]
100         if servers, ok := ck.config.Groups[gid]; ok {
101             ck.mu.Lock()
102             si := ck.leaderIds[gid]
103             ck.mu.Unlock()
104
105             for i := 0; i < len(servers); i++ {
106                 srv := ck.make_end(servers[si])
107                 var reply GetReply
108                 ok := srv.Call("ShardKV.Get", &args, &reply)
109                 if ok && (reply.Err == OK || reply.Err == ErrNoKey) {
110                     ck.mu.Lock()
111                     ck.leaderIds[gid] = si
112                     ck.mu.Unlock()
113                     if reply.Err == ErrNoKey {
114                         return ""
115                     }
116                     return reply.Value
117                 }
118                 if ok && reply.Err == ErrWrongGroup {
119                     break // 该 group 不负责此 shard, 刷新 config

```

```

120         }
121         // ErrWrongLeader / ErrTimeout / RPC 失败: 换下一台
122         si = (si + 1) % len(servers)
123     }
124 }
125 time.Sleep(100 * time.Millisecond)
126 ck.config = ck.sm.Query(-1)
127 }
128 }
129
130 // ===== PutAppend =====
131
132 func (ck *Clerk) PutAppend(key string, value string, op string) {
133     seq := ck.nextSeq()
134     args := PutAppendArgs{
135         Key:    key,
136         Value:  value,
137         Op:     op,
138         ClientId: ck.clientId,
139         SeqId:  seq,
140     }
141
142     for {
143         shard := key2shard(key)
144         gid := ck.config.Shards[shard]
145         if servers, ok := ck.config.Groups[gid]; ok {
146             ck.mu.Lock()
147             si := ck.leaderIds[gid]
148             ck.mu.Unlock()
149
150             for i := 0; i < len(servers); i++ {
151                 srv := ck.make_end(servers[si])
152                 var reply PutAppendReply
153                 ok := srv.Call("ShardKV.PutAppend", &args, &reply)
154                 if ok && reply.Err == OK {
155                     ck.mu.Lock()
156                     ck.leaderIds[gid] = si
157                     ck.mu.Unlock()
158                     return
159                 }
160                 if ok && reply.Err == ErrWrongGroup {
161                     break
162                 }
163                 si = (si + 1) % len(servers)
164             }
165         }
166         time.Sleep(100 * time.Millisecond)

```



```

167         ck.config = ck.sm.Query(-1)
168     }
169 }
170
171 func (ck *Clerk) Put(key string, value string) {
172     ck.PutAppend(key, value, "Put")
173 }
174
175 func (ck *Clerk) Append(key string, value string) {
176     ck.PutAppend(key, value, "Append")
177 }

```

server.go

```

1  package shardkv
2
3  // ===== 设计说明 =====
4  //
5  // Shard 状态机 (4 个状态) :
6  //
7  //   Serving   —config change—►   BePulling   (旧 owner: 停止服务, 等待新 owner
   拉取并 GC)
8  //   Serving   ◄—OpGC applied—   BePulling
9  //
10 //   (no shard) —config change—►   Pulling      (新 owner: 需要拉取数据)
11 //   Pulling   —OpInstall—►       GCing        (新 owner: 有数据可服务, 等待 GC 完
   成)
12 //   GCing     —OpGC applied—►   Serving      (新 owner: GC 已通知旧 owner, 稳定)
13 //
14 // Config 推进原则: 只有所有 shard 都处于稳定状态 (Serving) 才推进到下一个 config,
15 // 保证同一时刻只有一个 config 的迁移在进行。
16 //
17 // 幂等性:
18 //   - 客户端 op 通过 dupTable[clientId] 去重
19 //   - PullShard 数据连同 dupTable 一起迁移, 保证迁移后历史 op 依然不重复执行
20
21 import (
22     "bytes"
23     "sync"
24     "sync/atomic"
25     "time"
26
27     "6.824/labgob"
28     "6.824/labrpc"
29     "6.824/raft"
30     "6.824/shardctrler"

```

```

31 )
32
33 const ApplyTimeout = 500 * time.Millisecond
34
35 // ===== Shard 状态 =====
36
37 type ShardState int8
38
39 const (
40     Serving    ShardState = iota // 正常服务
41     Pulling      // 新 owner, 等待从旧 owner 拉取数据
42     BePulling    // 旧 owner, 数据被拉取中, 暂停服务
43     GCing        // 新 owner, 已有数据并服务, 等待通知旧 owner GC
44 )
45
46 type Shard struct {
47     State ShardState
48     KV     map[string]string
49 }
50
51 func newShard() Shard {
52     return Shard{State: Serving, KV: make(map[string]string)}
53 }
54
55 func (s *Shard) cloneKV() map[string]string {
56     m := make(map[string]string, len(s.KV))
57     for k, v := range s.KV {
58         m[k] = v
59     }
60     return m
61 }
62
63 // ===== Op 类型 =====
64
65 const (
66     OpGet    = "Get"
67     OpPut    = "Put"
68     OpAppend = "Append"
69     OpConfig = "Config"
70     OpInstall = "Install" // 安装迁移来的 shard 数据
71     OpGC      = "GC"      // 垃圾回收: 删除已迁移的 shard
72 )
73
74 type Op struct {
75     InternalId int64 // 用于 notifyChans 身份匹配 (所有 op 类型通用)
76
77     Type string

```

```

78     ClientId int64
79     SeqId    int64
80
81     // KV ops
82     Key    string
83     Value  string
84
85     // OpConfig
86     Config shardctrler.Config
87
88     // OpInstall
89     ConfigNum int
90     Shards    map[int]map[string]string
91     DupTable  map[int64]DupEntry
92
93     // OpGC
94     GCConfigNum int
95     GCShardIds  []int
96 }
97
98 // ===== applyResult =====
99
100 type applyResult struct {
101     InternalId int64
102     Err        Err
103     Value      string
104 }
105
106 // ===== notifyEntry =====
107
108 type notifyEntry struct {
109     internalId int64
110     ch         chan applyResult
111 }
112
113 // ===== ShardKV =====
114
115 type ShardKV struct {
116     mu      sync.Mutex
117     me      int
118     rf      *raft.Raft
119     applyCh chan raft.ApplyMsg
120     make_end func(string) *labrpc.ClientEnd
121     gid      int
122     ctrlers  []*labrpc.ClientEnd
123     maxraftstate int
124     persister *raft.Persister

```

```

125     dead          int32
126
127     mck            *shardctrler.Clerk
128     config         shardctrler.Config // 当前已 apply 的 config
129     lastConfig     shardctrler.Config // 上一个 config, 用于确定迁移来源 gid
130
131     shards         [shardctrler.NShards]Shard
132     dupTable       map[int64]DupEntry
133     notifyChans    map[int]notifyEntry
134 }
135
136 // ===== RPC Handler: Get =====
137
138 func (kv *ShardKV) Get(args *GetArgs, reply *GetReply) {
139     op := Op{
140         Type:    OpGet,
141         Key:     args.Key,
142         ClientId: args.ClientId,
143         SeqId:   args.SeqId,
144     }
145     err, value := kv.submitOp(op)
146     reply.Err = err
147     reply.Value = value
148 }
149
150 // ===== RPC Handler: PutAppend =====
151
152 func (kv *ShardKV) PutAppend(args *PutAppendArgs, reply *PutAppendReply) {
153     op := Op{
154         Type:    args.Op,
155         Key:     args.Key,
156         Value:   args.Value,
157         ClientId: args.ClientId,
158         SeqId:   args.SeqId,
159     }
160     err, _ := kv.submitOp(op)
161     reply.Err = err
162 }
163
164 // ===== RPC Handler: PullShard (被旧 owner 响应)
165 // =====
166 //
167 // 新 owner 的 migrationPuller 调用此 RPC 拉取 shard 数据。
168 // 只需是 leader 且 config num >= 请求的 config num 即可响应。
169 // 不走 Raft: 只读操作, leader 持有数据, 直接返回。
170
171 func (kv *ShardKV) PullShard(args *PullShardArgs, reply *PullShardReply) {

```

```

171     if _, isLeader := kv.rf.GetState(); !isLeader {
172         reply.Err = ErrWrongLeader
173         return
174     }
175
176     kv.mu.Lock()
177     defer kv.mu.Unlock()
178
179     if kv.config.Num < args.ConfigNum {
180         // 旧 owner 还没 apply 到对应的 config, 稍后重试
181         reply.Err = ErrNotReady
182         return
183     }
184
185     reply.Shards = make(map[int]map[string]string)
186     for _, shardId := range args.ShardIds {
187         reply.Shards[shardId] = kv.shards[shardId].cloneKV()
188     }
189
190     // 连同 dupTable 一起迁移, 保证幂等性历史不丢失
191     reply.DupTable = make(map[int64]DupEntry)
192     for k, v := range kv.dupTable {
193         reply.DupTable[k] = v
194     }
195
196     reply.Err = OK
197 }
198
199 // ===== RPC Handler: GCShard (被旧 owner 响应)
200 // =====
201 //
202 // 新 owner 拿到数据后, 通知旧 owner 删除。旧 owner 通过 Raft 保证所有副本一致删除。
203 func (kv *ShardKV) GCShard(args *GCArgs, reply *GCReply) {
204     if _, isLeader := kv.rf.GetState(); !isLeader {
205         reply.Err = ErrWrongLeader
206         return
207     }
208
209     kv.mu.Lock()
210     // 如果 config 已经推进到更新的版本, 说明 GC 已经完成 (幂等)
211     if kv.config.Num > args.ConfigNum {
212         kv.mu.Unlock()
213         reply.Err = OK
214         return
215     }
216     // 检查是否所有 shard 都还在 BePulling (如果已经是 Serving 说明已 GC)

```

```

217     allGCed := true
218     for _, shardId := range args.ShardIds {
219         if kv.shards[shardId].State == BePulling {
220             allGCed = false
221             break
222         }
223     }
224     if allGCed {
225         kv.mu.Unlock()
226         reply.Err = OK
227         return
228     }
229     kv.mu.Unlock()
230
231     op := Op{
232         Type:      OpGC,
233         GCConfigNum: args.ConfigNum,
234         GCShardIds:  args.ShardIds,
235     }
236     err, _ := kv.submitOp(op)
237     reply.Err = err
238 }
239
240 // ===== submitOp: 统一提交 + 等待 =====
241
242 func (kv *ShardKV) submitOp(op Op) (Err, string) {
243     op.InternalId = nrand()
244
245     index, _, isLeader := kv.rf.Start(op)
246     if !isLeader {
247         return ErrWrongLeader, ""
248     }
249
250     ch := make(chan applyResult, 1)
251
252     kv.mu.Lock()
253     if old, ok := kv.notifyChans[index]; ok {
254         // 同 index 有旧等待者 (leader 切换重分配了同一 index) , 通知它失败
255         old.ch <- applyResult{Err: ErrWrongLeader}
256     }
257     kv.notifyChans[index] = notifyEntry{internalId: op.InternalId, ch: ch}
258     kv.mu.Unlock()
259
260     defer func() {
261         kv.mu.Lock()
262         if e, ok := kv.notifyChans[index]; ok && e.internalId == op.InternalId

```

```

263         delete(kv.notifyChans, index)
264     }
265     kv.mu.Unlock()
266 }()
267
268 select {
269 case result := <-ch:
270     if result.InternalId != op.InternalId {
271         return ErrWrongLeader, ""
272     }
273     return result.Err, result.Value
274 case <-time.After(ApplyTimeout):
275     return ErrTimeout, ""
276 }
277 }
278
279 // ===== applier =====
280
281 func (kv *ShardKV) applier() {
282     for !kv.killed() {
283         msg := <-kv.applyCh
284
285         if msg.SnapshotValid {
286             kv.mu.Lock()
287             if kv.rf.CondInstallSnapshot(msg.SnapshotTerm, msg.SnapshotIndex,
msg.Snapshot) {
288                 kv.installSnapshot(msg.Snapshot)
289             }
290             kv.mu.Unlock()
291             continue
292         }
293
294         if !msg.CommandValid {
295             continue
296         }
297
298         op, ok := msg.Command.(Op)
299         if !ok {
300             continue
301         }
302
303         kv.mu.Lock()
304
305         var result applyResult
306         result.InternalId = op.InternalId
307
308         // 客户端 KV op: 走去重表

```

```

309         if op.Type == OpGet || op.Type == OpPut || op.Type == OpAppend {
310             shardId := key2shard(op.Key)
311             state := kv.shards[shardId].State
312             ownerGid := kv.config.Shards[shardId]
313
314             // 修复 Bug 1: 必须同时检查状态和所有权
315             if ownerGid != kv.gid || (state != Serving && state != GCing) {
316                 result.Err = ErrWrongGroup
317             } else if op.Type != OpGet {
318                 // 写操作去重
319                 if last, dup := kv.dupTable[op.ClientId]; dup && last.SeqId >=
op.SeqId {
320                     result.Err = last.Err
321                     result.Value = last.Value
322                 } else {
323                     result = kv.applyKVOp(op)
324                     result.InternalId = op.InternalId
325                     kv.dupTable[op.ClientId] = DupEntry{SeqId: op.SeqId, Err:
result.Err, Value: result.Value}
326                 }
327             } else {
328                 // Get: 不去重 (幂等) , 直接读
329                 result = kv.applyKVOp(op)
330                 result.InternalId = op.InternalId
331             }
332         } else {
333             // 管理 op: 直接执行 (配置变更、shard 迁移、GC)
334             result = kv.applyAdminOp(op)
335             result.InternalId = op.InternalId
336         }
337
338         // 取出 channel (锁外发送避免死锁)
339         var notifyCh chan applyResult
340         if e, ok := kv.notifyChans[msg.CommandIndex]; ok && e.internalId ==
op.InternalId {
341             notifyCh = e.ch
342         }
343
344         // 快照检查: 序列化在持锁时, rf.Snapshot() 在释放锁后调用, 避免 data race
345         if kv.maxraftstate != -1 && kv.persister.RaftStateSize() >=
kv.maxraftstate {
346             kv.doSnapshot(msg.CommandIndex)
347         }
348         kv.mu.Unlock()
349         if notifyCh != nil {
350             notifyCh <- result
351         }

```



```

352     }
353 }
354
355 // ===== KV 状态机操作 =====
356
357 func (kv *ShardKV) applyKVOp(op Op) applyResult {
358     shardId := key2shard(op.Key)
359     switch op.Type {
360     case OpGet:
361         v, ok := kv.shards[shardId].KV[op.Key]
362         if !ok {
363             return applyResult{Err: ErrNoKey}
364         }
365         return applyResult{Err: OK, Value: v}
366     case OpPut:
367         kv.shards[shardId].KV[op.Key] = op.Value
368         return applyResult{Err: OK}
369     case OpAppend:
370         kv.shards[shardId].KV[op.Key] += op.Value
371         return applyResult{Err: OK}
372     }
373     return applyResult{Err: ErrNoKey}
374 }
375
376 // ===== 管理 op 状态机 =====
377
378 func (kv *ShardKV) applyAdminOp(op Op) applyResult {
379     switch op.Type {
380
381     case OpConfig:
382         // 只推进 +1 的 config, 保证单步迁移
383         if op.Config.Num != kv.config.Num+1 {
384             return applyResult{Err: OK}
385         }
386         kv.lastConfig = kv.config
387         kv.config = op.Config
388
389         for i := 0; i < shardctrler.NShards; i++ {
390             nowOwner := op.Config.Shards[i] == kv.gid
391             wasOwner := kv.lastConfig.Shards[i] == kv.gid
392             prevGid := kv.lastConfig.Shards[i]
393
394             if nowOwner && !wasOwner {
395                 if prevGid == 0 {
396                     // Config 1: 从无到有, 直接 Serving
397                     kv.shards[i].State = Serving
398                 } else {

```

```

399         // 需要从旧 owner 拉取
400         kv.shards[i].State = Pulling
401     }
402 }
403 if !nowOwner && wasOwner {
404     // 旧 owner: 等待新 owner 拉取, 暂停服务
405     kv.shards[i].State = BePulling
406 }
407 }
408 return applyResult{Err: OK}
409
410 case OpInstall:
411     // 只安装与当前 config 匹配的数据
412     if op.ConfigNum != kv.config.Num {
413         return applyResult{Err: OK}
414     }
415     for shardId, kvData := range op.Shards {
416         if kv.shards[shardId].State == Pulling {
417             newKV := make(map[string]string, len(kvData))
418             for k, v := range kvData {
419                 newKV[k] = v
420             }
421             kv.shards[shardId].KV = newKV
422             kv.shards[shardId].State = GCing // 有数据可服务, 但还没 GC 旧
owner
423         }
424     }
425     // 合并 dupTable: 取较新的 SeqId
426     for clientId, entry := range op.DupTable {
427         if existing, ok := kv.dupTable[clientId]; !ok || entry.SeqId >
existing.SeqId {
428             kv.dupTable[clientId] = entry
429         }
430     }
431     return applyResult{Err: OK}
432
433 case OpGC:
434     // 旧 owner: BePulling → Serving (删数据)
435     // 新 owner: GCing → Serving (稳定)
436     for _, shardId := range op.GCShardIds {
437         switch kv.shards[shardId].State {
438             case BePulling:
439                 if kv.config.Num == op.GCConfigNum {
440                     kv.shards[shardId].KV = make(map[string]string)
441                     kv.shards[shardId].State = Serving
442                 }
443             case GCing:

```

```

444         if kv.config.Num == op.GCConfigNum {
445             kv.shards[shardId].State = Serving
446         }
447     }
448 }
449 return applyResult{Err: OK}
450 }
451
452 return applyResult{Err: OK}
453 }
454
455 // ===== 后台 goroutine: config 轮询 =====
456 //
457 // 只有当前所有 shard 都处于 Serving 状态时才推进 config。
458 // 保证同一时刻只有一个 config 的迁移在进行。
459
460 func (kv *ShardKV) configPoller() {
461     for !kv.killed() {
462         if _, isLeader := kv.rf.GetState(); isLeader {
463             kv.mu.Lock()
464             allStable := true
465             for _, shard := range kv.shards {
466                 if shard.State != Serving {
467                     allStable = false
468                     break
469                 }
470             }
471             currentNum := kv.config.Num
472             kv.mu.Unlock()
473
474             if allStable {
475                 newCfg := kv.mck.Query(currentNum + 1)
476                 if newCfg.Num == currentNum+1 {
477                     kv.rf.Start(Op{
478                         Type:      OpConfig,
479                         Config:    newCfg,
480                         InternalId: nrand(),
481                     })
482                 }
483             }
484         }
485         time.Sleep(100 * time.Millisecond)
486     }
487 }
488
489 // ===== 后台 goroutine: shard 拉取 =====
490 //

```

```

491 // 收集所有 Pulling 状态的 shard, 按 gid 分组,
492 // 向上一个 config 中的旧 owner 发送 PullShard RPC,
493 // 成功后提交 OpInstall 到 Raft。
494
495 func (kv *ShardKV) migrationPuller() {
496     for !kv.killed() {
497         if _, isLeader := kv.rf.GetState(); isLeader {
498             kv.mu.Lock()
499             pullMap := make(map[int][]int) // gid -> []shardId
500             for i, shard := range kv.shards {
501                 if shard.State == Pulling {
502                     gid := kv.lastConfig.Shards[i]
503                     pullMap[gid] = append(pullMap[gid], i)
504                 }
505             }
506             configNum := kv.config.Num
507             lastCfg := kv.lastConfig
508             kv.mu.Unlock()
509
510             var wg sync.WaitGroup
511             for gid, shardIds := range pullMap {
512                 wg.Add(1)
513                 go func(gid int, shardIds []int) {
514                     defer wg.Done()
515                     servers, ok := lastCfg.Groups[gid]
516                     if !ok {
517                         return
518                     }
519                     args := PullShardArgs{ConfigNum: configNum, ShardIds:
shardIds}
520
521                     for _, server := range servers {
522                         srv := kv.make_end(server)
523                         var reply PullShardReply
524                         if srv.Call("ShardKV.PullShard", &args, &reply) &&
reply.Err == OK {
525                             kv.rf.Start(Op{
526                                 Type:      OpInstall,
527                                 InternalId: nrand(),
528                                 ConfigNum: configNum,
529                                 Shards:    reply.Shards,
530                                 DupTable:  reply.DupTable,
531                             })
532                             return
533                         }
534                     }(gid, shardIds)
535                 }

```



```

582             })
583             return
584         }
585     }
586     }(gid, shardIds)
587 }
588 wg.Wait()
589 }
590 time.Sleep(50 * time.Millisecond)
591 }
592 }
593
594 // ===== 快照 =====
595
596 func (kv *ShardKV) doSnapshot(index int) {
597     w := new(bytes.Buffer)
598     e := labgob.NewEncoder(w)
599     e.Encode(kv.shards)
600     e.Encode(kv.dupTable)
601     e.Encode(kv.config)
602     e.Encode(kv.lastConfig)
603     kv.rf.Snapshot(index, w.Bytes())
604 }
605
606 func (kv *ShardKV) installSnapshot(data []byte) {
607     if len(data) == 0 {
608         return
609     }
610     r := bytes.NewBuffer(data)
611     d := labgob.NewDecoder(r)
612     var shards [shardctrler.NShards]Shard
613     var dupTable map[int64]DupEntry
614     var config shardctrler.Config
615     var lastConfig shardctrler.Config
616     if d.Decode(&shards) != nil ||
617         d.Decode(&dupTable) != nil ||
618         d.Decode(&config) != nil ||
619         d.Decode(&lastConfig) != nil {
620         return
621     }
622     kv.shards = shards
623     kv.dupTable = dupTable
624     kv.config = config
625     kv.lastConfig = lastConfig
626 }
627
628 // ===== Kill =====

```

```

629
630 func (kv *ShardKV) Kill() {
631     atomic.StoreInt32(&kv.dead, 1)
632     kv.rf.Kill()
633 }
634
635 func (kv *ShardKV) killed() bool {
636     return atomic.LoadInt32(&kv.dead) == 1
637 }
638
639 // ===== StartServer =====
640
641 func StartServer(servers []*labrpc.ClientEnd, me int, persister
    *raft.Persister,
642     maxraftstate int, gid int, ctrlers []*labrpc.ClientEnd,
643     make_end func(string) *labrpc.ClientEnd) *ShardKV {
644
645     // 注册所有需要 gob 序列化的类型
646     labgob.Register(Op{})
647     labgob.Register(shardctrler.Config{})
648     labgob.Register(map[int]map[string]string{})
649     labgob.Register(map[int64]DupEntry{})
650     labgob.Register(Shard{})
651
652     kv := &ShardKV{
653         me:          me,
654         make_end:     make_end,
655         gid:          gid,
656         ctrlers:      ctrlers,
657         maxraftstate: maxraftstate,
658         persister:    persister,
659         mck:          shardctrler.MakeClerk(ctrlers),
660         dupTable:     make(map[int64]DupEntry),
661         notifyChans:  make(map[int]notifyEntry),
662     }
663
664     // 初始化所有 shard 为 Serving 状态 (空数据)
665     for i := range kv.shards {
666         kv.shards[i] = newShard()
667     }
668
669     kv.applyCh = make(chan raft.ApplyMsg, 64)
670     kv.rf = raft.Make(servers, me, persister, kv.applyCh)
671
672     // 从快照恢复
673     if snapshot := persister.ReadSnapshot(); len(snapshot) > 0 {
674         kv.installSnapshot(snapshot)

```

```
675     }
676
677     go kv.applier()
678     go kv.configPoller()
679     go kv.migrationPuller()
680     go kv.gcHandler()
681
682     return kv
683 }
```